

Binární vyhledávací strom

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

10.06.2026

CC BY-NC-SA 4.0

Binární vyhledávací strom

Binární vyhledávací strom

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

Pro **každý** uzel platí:

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

Pro **každý** uzel platí:

- Všechny hodnoty v **levém** podstromu jsou **menší**

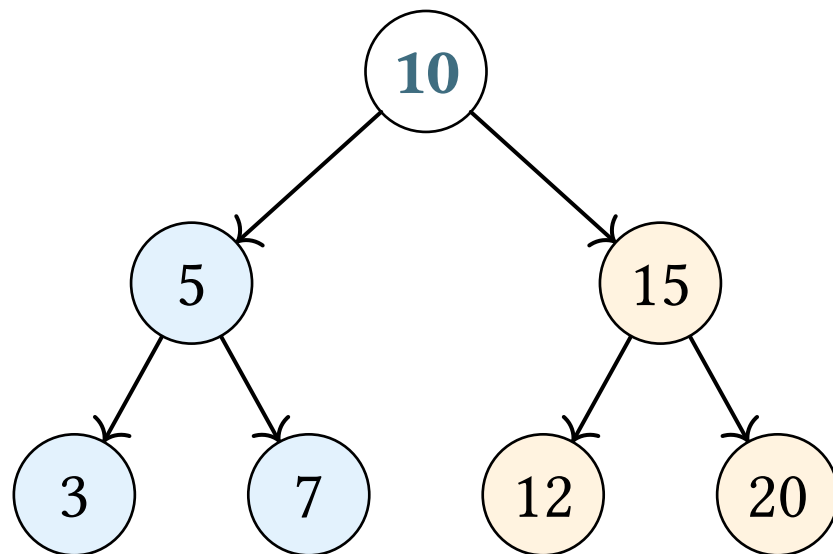
Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

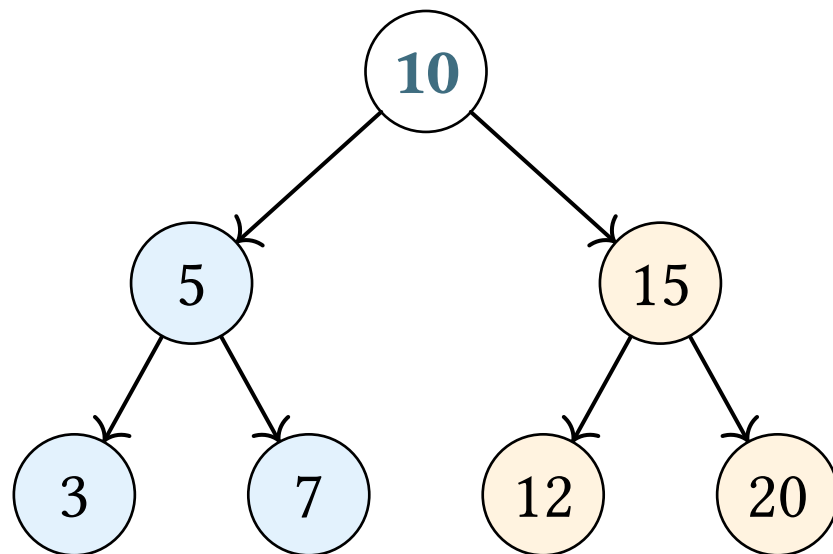
Pro **každý** uzel platí:

- Všechny hodnoty v **levém** podstromu jsou **menší**
- Všechny hodnoty v **pravém** podstromu jsou **větší**

Binární vyhledávací strom



Binární vyhledávací strom



Modré < 10

Oranžové > 10

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

BST umožňuje **efektivně vkládat i mazat**, na rozdíl od pole

Operace na BST

Vyhledávání

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**
5. Je-li uzel NULL →

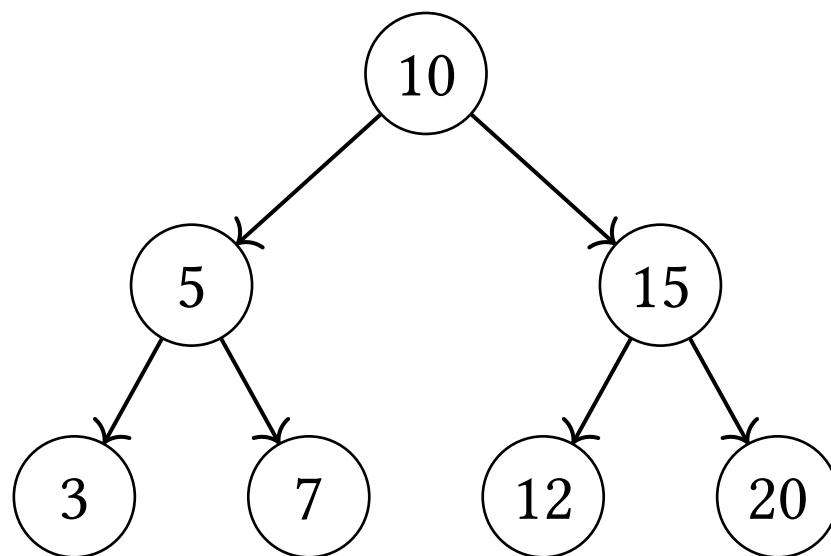
Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**
5. Je-li uzel NULL → **nenalezeno**

Vyhledávání

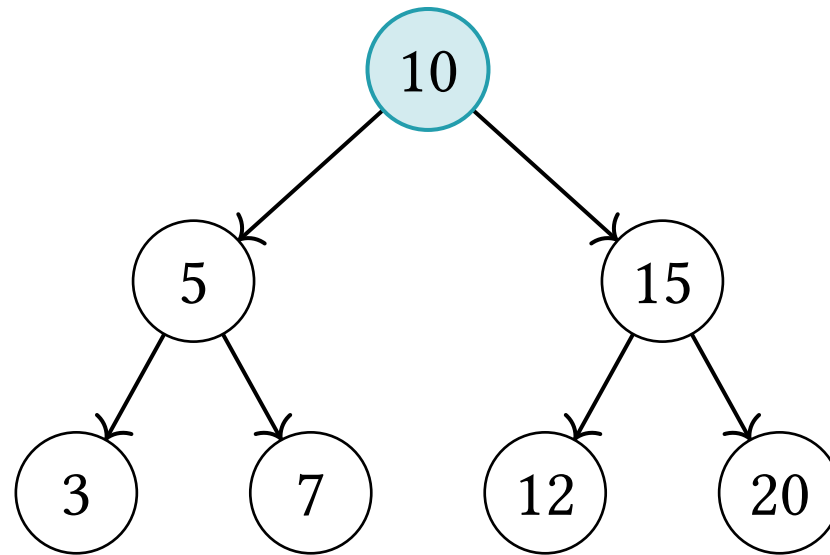
Hledáme hodnotu 12:



Začínáme u kořene

Vyhledávání

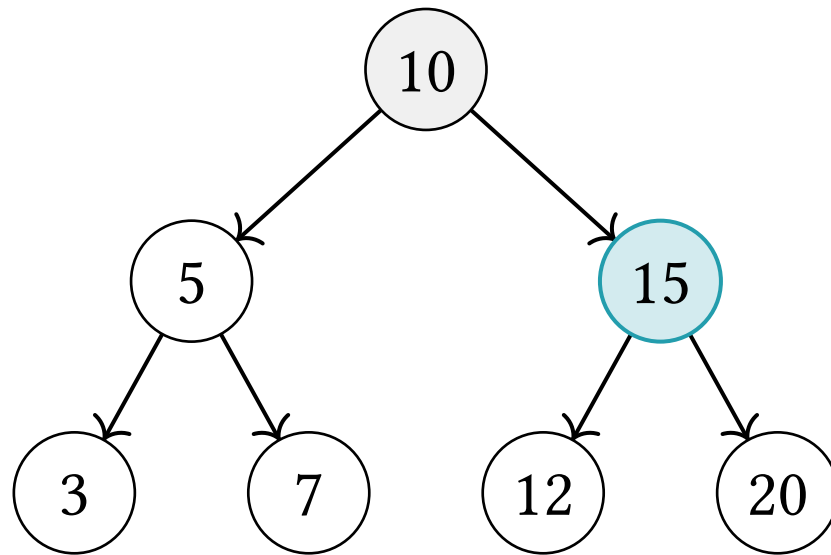
Hledáme hodnotu 12:



$12 > 10 \rightarrow$ jdeme **doprava**

Vyhledávání

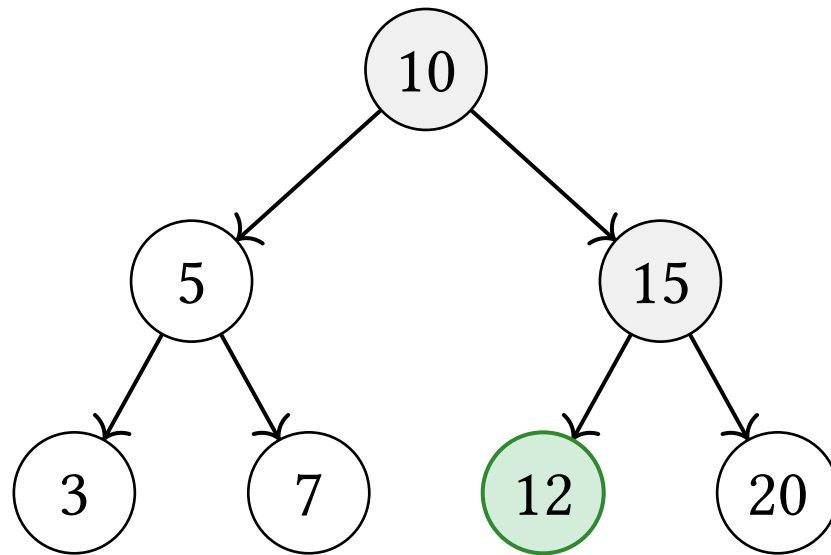
Hledáme hodnotu 12:



$12 < 15 \rightarrow$ jdeme **doleva**

Vyhledávání

Hledáme hodnotu 12:



$12 == 12 \rightarrow$ **nalezeno!**

Vyhledávání

Jakou složitost má vyhledávání v BST?

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Degenerovaný strom: $h = n \rightarrow$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Degenerovaný strom: $h = n \rightarrow \mathcal{O}(n)$

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

$$n = 2^0 + 2^1 + \dots + 2^{h-1}$$

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

$$n = 2^0 + 2^1 + \dots + 2^{h-1}$$

Použijeme vzorec pro součet geometrické řady, který už známe z přednášky o amortizované složitosti ($q = 2$):

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

$$n = 2^0 + 2^1 + \dots + 2^{h-1}$$

Použijeme vzorec pro součet geometrické řady, který už známe z přednášky o amortizované složitosti ($q = 2$):

$$n = \frac{2^h - 1}{2 - 1} = 2^h - 1$$

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

$$n = 2^0 + 2^1 + \dots + 2^{h-1}$$

Použijeme vzorec pro součet geometrické řady, který už známe z přednášky o amortizované složitosti ($q = 2$):

$$n = \frac{2^h - 1}{2 - 1} = 2^h - 1 \rightarrow 2^h = n + 1$$

Odvození výšky vyváženého stromu

Na každé úrovni i je nejvýše 2^i uzlů:

$$n = 2^0 + 2^1 + \dots + 2^{h-1}$$

Použijeme vzorec pro součet geometrické řady, který už známe z přednášky o amortizované složitosti ($q = 2$):

$$n = \frac{2^h - 1}{2 - 1} = 2^h - 1 \rightarrow 2^h = n + 1$$

$$h = \log_2(n + 1) \rightarrow \mathcal{O}(\log n)$$

Vkládání

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání
2. Když narazíš na NULL →

Vkládání

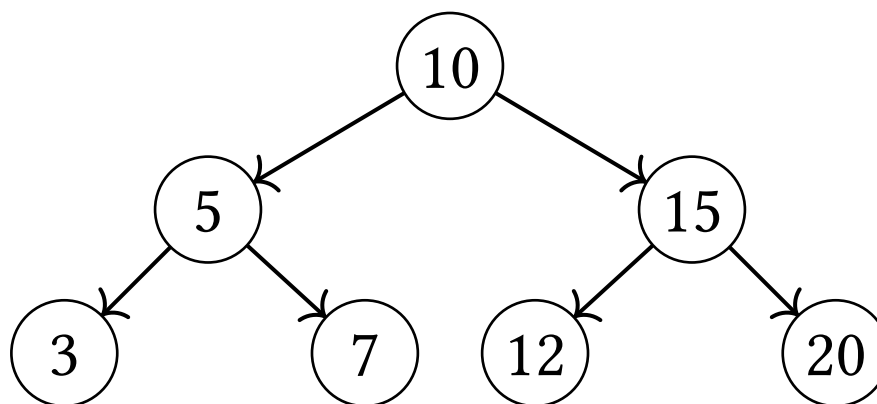
Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání
2. Když narazíš na NULL → vlož nový uzel na toto místo

Vkládání

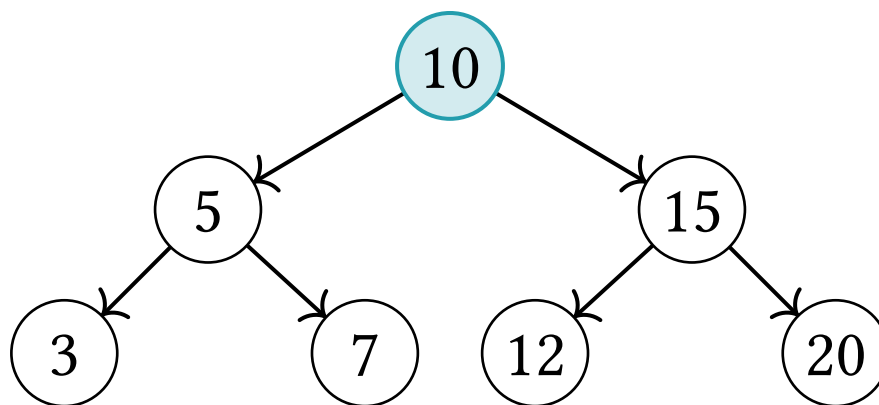
Vkládáme hodnotu 13:



Začínáme u kořene

Vkládání

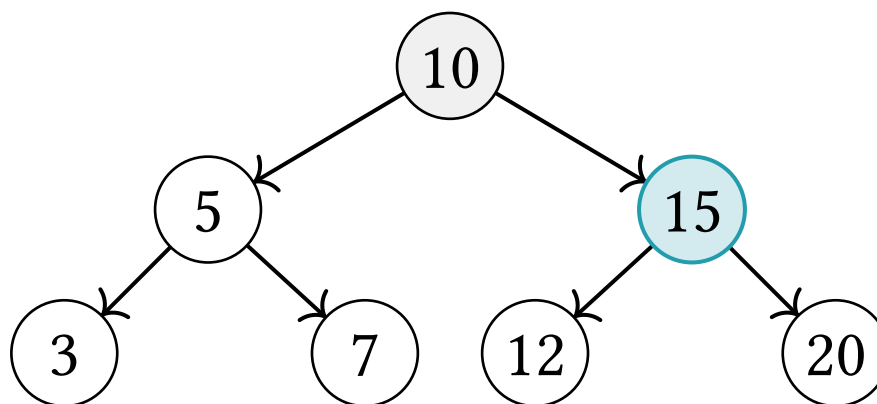
Vkládáme hodnotu 13:



$13 > 10 \rightarrow$ jdeme **doprava**

Vkládání

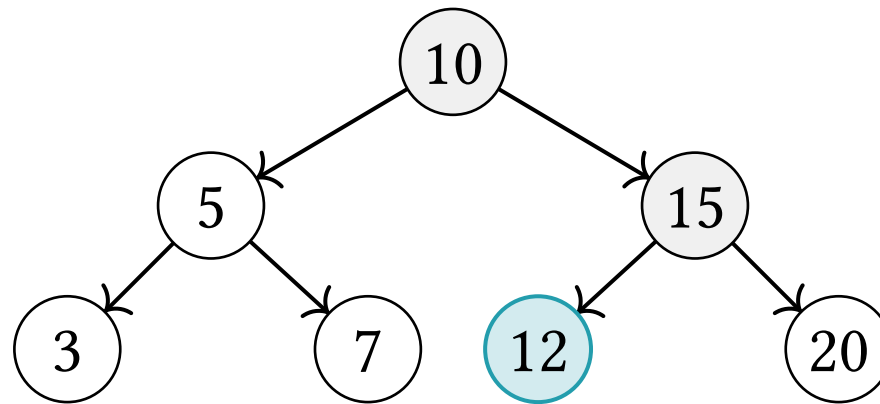
Vkládáme hodnotu 13:



$13 < 15 \rightarrow$ jdeme **doleva**

Vkládání

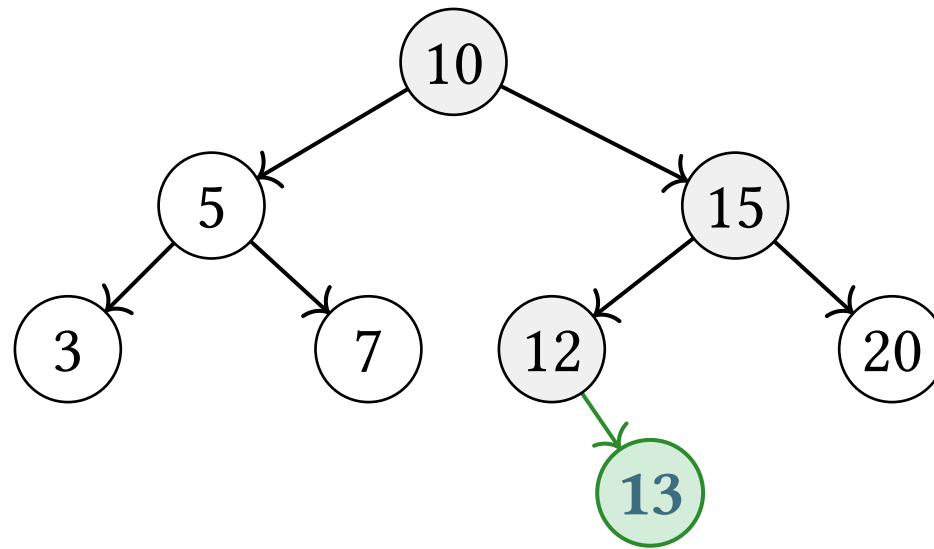
Vkládáme hodnotu 13:



$13 > 12 \rightarrow$ jdeme **doprava** $\rightarrow$ NULL

Vkládání

Vkládáme hodnotu 13:



Vložíme **13** jako pravého potomka uzlu 12

Vkládání

Jakou složitost má vkládání do BST?

Vkládání

Jakou složitost má vkládání do BST?

Také $\mathcal{O}(h)$, protože vkládání prochází stromem stejně jako vyhledávání

Mazání

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky)

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky) → jednoduše ho odstraníme

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky) → jednoduše ho odstraníme
2. Mazaný uzel má **jednoho** potomka

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky) → jednoduše ho odstraníme
2. Mazaný uzel má **jednoho** potomka → nahradíme ho jeho potomkem

Mazání

Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky) → jednoduše ho odstraníme
2. Mazaný uzel má **jednoho** potomka → nahradíme ho jeho potomkem
3. Mazaný uzel má **dva** potomky

Mazání

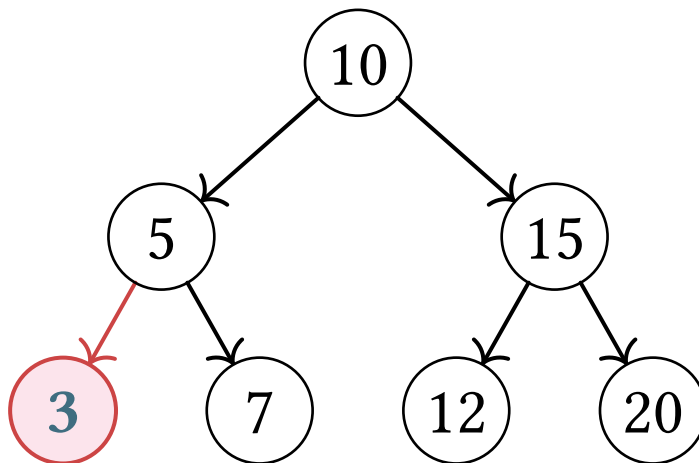
Mazání z BST je složitější – záleží na počtu potomků mazaného uzlu:

1. Mazaný uzel je **list** (nemá potomky) → jednoduše ho odstraníme
2. Mazaný uzel má **jednoho** potomka → nahradíme ho jeho potomkem
3. Mazaný uzel má **dva** potomky → nahradíme ho **in-order následníkem**

Mazání

Případ 1 – mazaný uzel je list

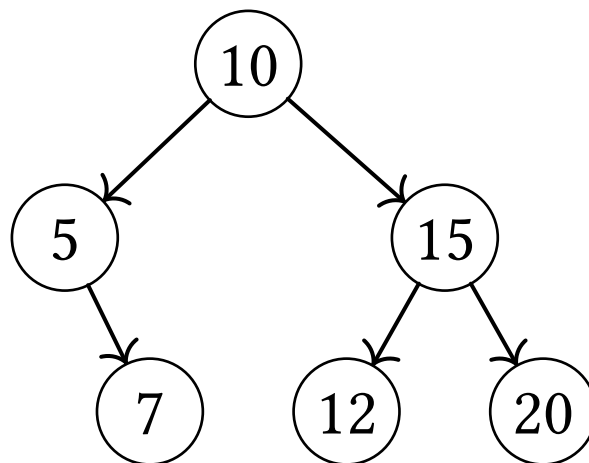
Smažeme uzel **3** – nemá žádné potomky:



Mazání

Případ 1 – mazaný uzel je list

Smažeme uzel **3** – nemá žádné potomky:

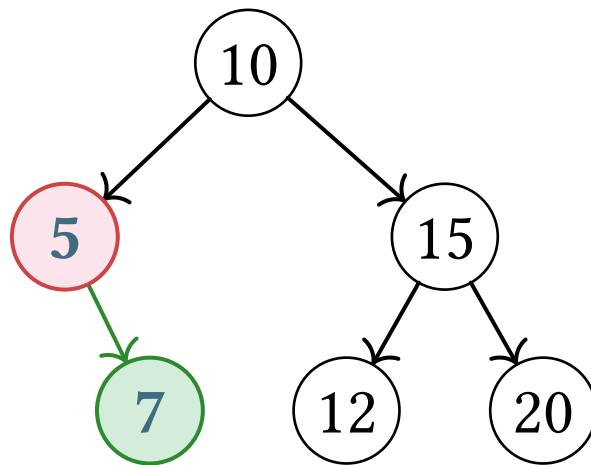


Jednoduše uzel odstraníme

Mazání

Případ 2 – mazaný uzel má jednoho potomka

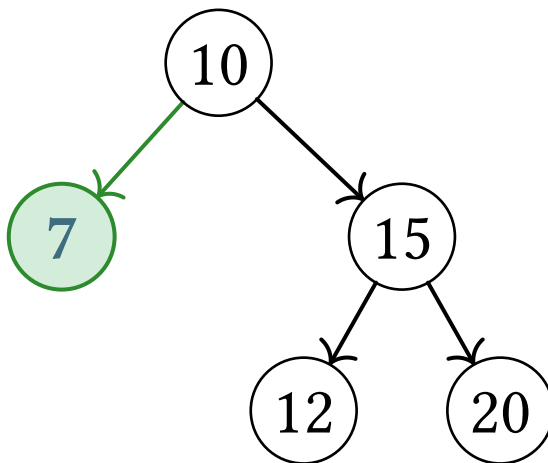
Po smazání uzlu 3 má uzel 5 jen pravého potomka – smažeme uzel 5:



Mazání

Případ 2 – mazaný uzel má jednoho potomka

Po smazání uzlu 3 má uzel 5 jen pravého potomka – smažeme uzel 5:



Nahradíme mazaný uzel jeho jediným potomkem

Mazání

Případ 3 – mazaný uzel má dva potomky

Mazání

Případ 3 – mazaný uzel má dva potomky

Potřebujeme najít **náhradu**, která zachová vlastnost BST

Mazání

Případ 3 – mazaný uzel má dva potomky

Potřebujeme najít **náhradu**, která zachová vlastnost BST

In-order následník = nejmenší prvek v pravém podstromu

Mazání

Případ 3 – mazaný uzel má dva potomky

Potřebujeme najít **náhradu**, která zachová vlastnost BST

In-order následník = nejmenší prvek v pravém podstromu (nejlevější uzel)

Mazání

Případ 3 – mazaný uzel má dva potomky

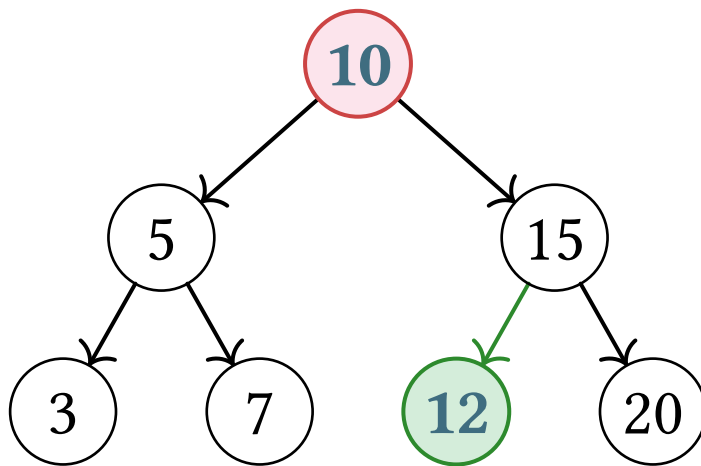
Potřebujeme najít **náhradu**, která zachová vlastnost BST

In-order následník = nejmenší prvek v pravém podstromu (nejlevější uzel)

Tento uzel je vždy **větší** než vše vlevo a **menší** než vše ostatní vpravo – ideální náhrada

Mazání

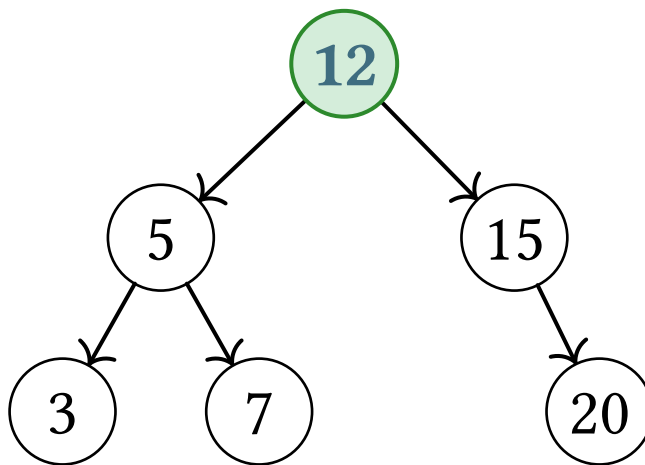
Smažeme **kořen 10** – má dva potomky:



In-order následník uzlu 10: jdi **doprava** (15), pak **co nejvíc doleva** (12)

Mazání

Smažeme **kořen 10** – má dva potomky:



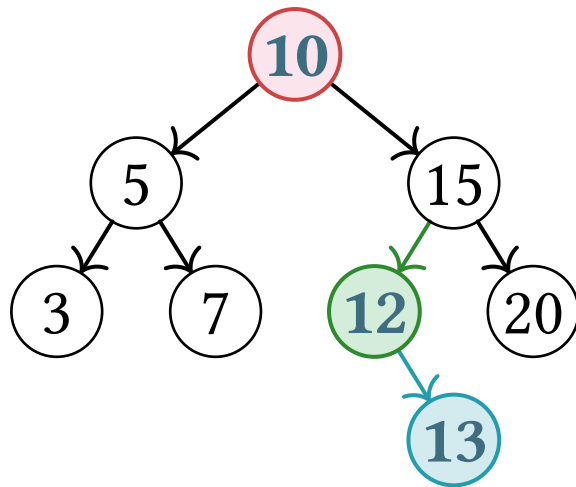
Nahradíme uzel 10 jeho in-order následníkem 12

Mazání

Co když má následník pravého potomka?

Mazání

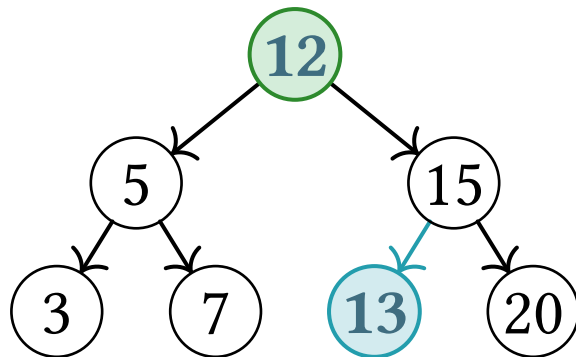
Co když má následník pravého potomka? Smažeme **10**, následník **12** má potomka **13**:



12 nahradí kořen, **13** zaujme místo **12**

Mazání

Co když má následník **pravého** potomka? Smažeme **10**, následník **12** má potomka **13**:



Následník nemůže mít **levého** potomka, ale **pravý** se posune na jeho místo

Mazání

Jakou složitost má mazání z BST?

Mazání

Jakou složitost má mazání z BST?

Také $\mathcal{O}(h)$ – musíme najít uzel a případně najít in-order následníka

Mazání

Jakou složitost má mazání z BST?

Také $\mathcal{O}(h)$ – musíme najít uzel a případně najít in-order následníka

Obojí vyžaduje průchod maximálně do hloubky stromu

Degenerovaný strom

Degenerovaný strom

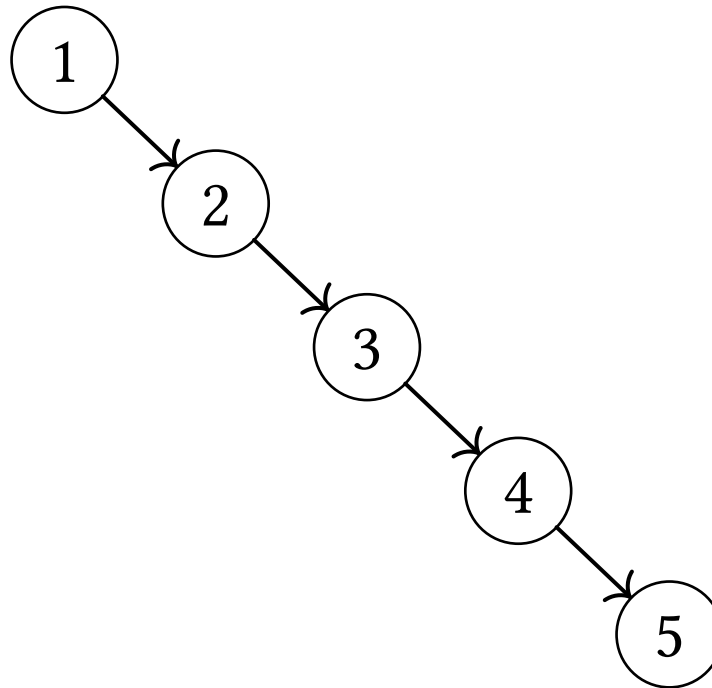
Co se stane, když vložíme prvky **v seřazeném pořadí**?

Degenerovaný strom

Co se stane, když vložíme prvky **v seřazeném pořadí**?

Vložíme postupně: 1, 2, 3, 4, 5

Degenerovaný strom



Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je

Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je $\mathcal{O}(n)$

Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je $\mathcal{O}(n)$

Řešením jsou **samovyvažovací stromy** (AVL, Red-Black)

AVL si ukážeme na příští přednášce

Shrnutí

Shrnutí

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Mazání: tři případy – list, jeden potomek, dva potomky (in-order následník)

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Mazání: tři případy – list, jeden potomek, dva potomky (in-order následník)
- Pro vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Mazání: tři případy – list, jeden potomek, dva potomky (in-order následník)
- Pro vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Mazání: tři případy – list, jeden potomek, dva potomky (in-order následník)
- Pro vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$
- BST je **dynamická** varianta binárního vyhledávání v seřazeném poli

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání, vkládání i mazání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Mazání: tři případy – list, jeden potomek, dva potomky (in-order následník)
- Pro vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$
- BST je **dynamická** varianta binárního vyhledávání v seřazeném poli
- Řešením degenerace jsou samovyvažovací stromy (AVL, Red-Black)